1. Morphological Processing: Analyzing, Analysis, Analytical

```python
words = ["analyzing", "analysis", "analytical"]
rules = {
    "analyzing": {
        "root": "analyze",
        "prefix": "None",
        "suffix": "ing",
        "type": "Inflectional",
        "normalized": "analyze"
    },
    "analysis": {
        "root": "analy",
        "prefix": "None",
        "suffix": "sis",
        "type": "Derivational / Irregular form",
        "normalized": "analyze"
    },
    "analytical": {
        "root": "analysis",
        "prefix": "None",
        "suffix": "tical",
        "type": "Derivational",
        "normalized": "analyze"
    }
}
print("MORPHOLOGICAL PROCESSING REPORT")
print("-" * 90)
for word in words:
    data = rules[word]
    print("\nOriginal Word:", word)
    print("Root Word:", data["root"])
    print("Prefix:", data["prefix"])
    print("Suffix:", data["suffix"])
    print("Transformation Type:", data["type"])
    print("Normalized Output:", data["normalized"])
```

Output

```
MORPHOLOGICAL PROCESSING REPORT
------------------------------------------------------------------------------------------

Original Word: analyzing
Root Word: analyze
Prefix: None
Suffix: ing
Transformation Type: Inflectional
Normalized Output: analyze

Original Word: analysis
Root Word: analy
Prefix: None
Suffix: sis
Transformation Type: Derivational / Irregular form
Normalized Output: analyze

Original Word: analytical
Root Word: analysis
Prefix: None
Suffix: tical
Transformation Type: Derivational
Normalized Output: analyze
```

2. Morphological Parser: Disagree, Agreement, Agreeable

```
words = ["disagree", "agreement", "agreeable"]
rules = {
    "disagree": {
        "prefix": "dis",
        "root": "agree",
        "suffix": "None",
        "type": "Derivational",
        "meaning": "Express the opposite of agree",
        "normalized": "agree"
    },
    "agreement": {
        "prefix": "None",
        "root": "agree",
        "suffix": "ment",
        "type": "Derivational",
        "meaning": "The state or result of agreeing",
        "normalized": "agree"
    },
    "agreeable": {
        "prefix": "None",
        "root": "agree",
        "suffix": "able",
        "type": "Derivational",
        "meaning": "Pleasant or acceptable",
        "normalized": "agree"
    }
}
print("MORPHOLOGICAL PARSING REPORT")
print("-" * 80)
for word in words:
    data = rules[word]
    print("\nOriginal Word:", word)
    print("Prefix:", data["prefix"])
    print("Root Word:", data["root"])
    print("Suffix:", data["suffix"])
    print("Transformation Type:", data["type"])
```

Output

```
MORPHOLOGICAL PARSING REPORT
--------------------------------------------------------------------------------

Original Word: disagree
Prefix: dis
Root Word: agree
Suffix: None
Transformation Type: Derivational
Semantic Interpretation: Express the opposite of agree
Normalized Base Form: agree

Original Word: agreement
Prefix: None
Root Word: agree
Suffix: ment
Transformation Type: Derivational
Semantic Interpretation: The state or result of agreeing
Normalized Base Form: agree

Original Word: agreeable
Prefix: None
Root Word: agree
Suffix: able
Transformation Type: Derivational
Semantic Interpretation: Pleasant or acceptable
Normalized Base Form: agree
```

3. Morphology-Based Normalization: Govern, Government, Governance

```
words = ["govern", "government", "governance"]
rules = {
    "govern": {
        "root": "govern",
        "suffix": "None",
        "level": 0,
        "normalized": "govern"
    },
    "government": {
        "root": "govern",
        "suffix": "ment",
        "level": 1,
        "normalized": "govern"
    },
    "governance": {
        "root": "govern",
        "suffix": "ance",
        "level": 1,
        "normalized": "govern"
    }
}
print("MORPHOLOGY-BASED NORMALIZATION REPORT")
print("-" * 80)
for word in words:
    data = rules[word]
    print("\nOriginal Word:", word)
    print("Root Form:", data["root"])
    print("Detected Suffix:", data["suffix"])
    print("Derivational Level:", data["level"])
    print("Normalized Representation:", data["normalized"])
    print("Final Output for Topic Modeling:", data["normalized"])
```

Output

```
MORPHOLOGY-BASED NORMALIZATION REPORT
--------------------------------------------------------------------------------

Original Word: govern
Root Form: govern
Detected Suffix: None
Derivational Level: 0
Normalized Representation: govern
Final Output for Topic Modeling: govern

Original Word: government
Root Form: govern
Detected Suffix: ment
Derivational Level: 1
Normalized Representation: govern
Final Output for Topic Modeling: govern

Original Word: governance
Root Form: govern
Detected Suffix: ance
Derivational Level: 1
Normalized Representation: govern
Final Output for Topic Modeling: govern
```

4. Morphological Parsing: Activate, Activation, Reactivation

```python
words = ["activate", "activation", "reactivation"]
rules = {
    "activate": {
        "prefix": "None",
        "root": "act",
        "suffix": "ivate",
        "sequence": "act -> activate",
        "word_class": "Verb",
        "meaning": "To make something active",
        "normalized": "activate"
    },
    "activation": {
        "prefix": "None",
        "root": "act",
        "suffix": "ivation",
        "sequence": "act -> activate -> activation",
        "word_class": "Noun",
        "meaning": "The process of making something active",
        "normalized": "activate"
    },
    "reactivation": {
        "prefix": "re",
        "root": "act",
        "suffix": "ivation",
        "sequence": "act -> activate -> reactivate -> reactivation",
        "word_class": "Noun",
        "meaning": "The process of making something active again",
        "normalized": "activate"
    }
}
print("MORPHOLOGICAL PARSING AND NORMALIZATION")
print("-" * 80)
for word in words:
    data = rules[word]
    print("\nOriginal Word:", word)
    print("Prefix:", data["prefix"])
```

Output

```
MORPHOLOGICAL PARSING AND NORMALIZATION
--------------------------------------------------------------------------------

Original Word: activate
Prefix: None
Root Word: act
Suffix: ivate
Derivational Sequence: act -> activate
Word Class: Verb
Semantic Meaning: To make something active
Normalized Base Form: activate

Original Word: activation
Prefix: None
Root Word: act
Suffix: ivation
Derivational Sequence: act -> activate -> activation
Word Class: Noun
Semantic Meaning: The process of making something active
Normalized Base Form: activate

Original Word: reactivation
Prefix: re
Root Word: act
Suffix: ivation
Derivational Sequence: act -> activate -> reactivate -> reactivation
Word Class: Noun
Semantic Meaning: The process of making something active again
Normalized Base Form: activate
```

5. Inflectional Morphology: Create, Creates, Creating

```
words = ["create", "creates", "creating"]
rules = {
    "create": {
        "root": "create",
        "suffix": "None",
        "category": "Base Form",
        "normalized": "create"
    },
    "creates": {
        "root": "create",
        "suffix": "s",
        "category": "Third-Person Singular Present",
        "normalized": "create"
    },
    "creating": {
        "root": "create",
        "suffix": "ing",
        "category": "Present Participle",
        "normalized": "create"
    }
}
print("INFLECTIONAL MORPHOLOGY REPORT")
print("-" * 80)
for word in words:
    data = rules[word]
    print("\nOriginal Word:", word)
    print("Identified Suffix:", data["suffix"])
    print("Grammatical Category:", data["category"])
    print("Extracted Root:", data["root"])
    print("Normalized Base Form:", data["normalized"])
    print("Final Normalized Representation:", data["normalized"])
```

Output

```
INFLECTIONAL MORPHOLOGY REPORT
--------------------------------------------------------------------------------

Original Word: create
Identified Suffix: None
Grammatical Category: Base Form
Extracted Root: create
Normalized Base Form: create
Final Normalized Representation: create

Original Word: creates
Identified Suffix: s
Grammatical Category: Third-Person Singular Present
Extracted Root: create
Normalized Base Form: create
Final Normalized Representation: create

Original Word: creating
Identified Suffix: ing
Grammatical Category: Present Participle
Extracted Root: create
Normalized Base Form: create
Final Normalized Representation: create
```